Wayne's Clinic Baltimore - Advanced Database Systems

Portfolio 3 — Transactions and Concurrency

Organisation: Wayne's Clinic Baltimore · **Engine:** PostgreSQL 16 (MVCC, Serializable Snapshot Isolation)

A hospital database is written by many people at once. Receptionists book, doctors diagnose, pharmacists dispense and the lab files results. This portfolio shows, on a real server, what transactions guarantee, what goes wrong without the right isolation or locking, and which mechanism fixes each problem. Every demonstration opens genuine concurrent sessions, each a separate PostgreSQL backend process, and interleaves their statements in a fixed order so the outcome is reproducible.

Deliverable	Where
Commit, rollback, savepoint, atomicity on error	§2 · `sql/01_commit_rollback_savepoint.sql`
Isolation levels (dirty / non-repeatable / phantom reads)	§3 · C1–C3
Concurrency problems and control mechanisms	§4 · C4–C7 (lost update, write skew, double booking, deadlock)
MVCC and durability	§5 · C8, C9
Screenshots / output evidence	`evidence/`: a `.txt` log with millisecond timestamps and a two-column timeline `.png` per demo

1. Transactions in one paragraph

A transaction is a unit of work that the database treats as indivisible. **ACID** names its guarantees. *Atomicity*: all of its changes happen or none do. *Consistency*: it moves the database from one state that satisfies every constraint to another. *Isolation*: concurrent transactions do not see each other's half-finished work, to a degree set by the isolation level. *Durability*: once `COMMIT` returns, the change survives a crash. PostgreSQL implements isolation with **multi-version concurrency control (MVCC)**: an update writes a new row version instead of overwriting, and each transaction reads from a *snapshot*. Readers therefore never block writers and writers never block readers. Only writers competing for the same row wait for each other.

2. Commit, rollback and savepoints (single session)

Full output: `evidence/01_commit_rollback_savepoint_output.txt`.

Demo	Scenario	What it proves
T1 COMMIT	A walk-in patient is registered *and* booked in one transaction. After `COMMIT`, a brand-new connection sees both rows, stamped with the creating transaction id (`xmin`).	Committed work is visible to everyone and permanent
T2 ROLLBACK	A pharmacist removes 100 insulin pens instead of 10 and also writes an audit row, then cancels.	`ROLLBACK` undoes *every* statement in the transaction, the audit row included
T3 Atomicity on error	James Holloway walks into the Emergency Department on a Sunday. The doctor stores a diagnosis, then prescribes penicillin, to which he is allergic. The trigger raises `WC001`. The transaction enters the *aborted* state; even `SELECT` is refused (`25P02`) and `COMMIT` becomes `ROLLBACK`.	The diagnosis written *before* the error is discarded too: all or nothing
T4 SAVEPOINT	Same consultation with `SAVEPOINT before_line_1`. After the allergy error, `ROLLBACK TO SAVEPOINT` undoes only the bad line; azithromycin is	Partial rollback keeps valid work inside a long transaction

Demo	Scenario	What it proves
	prescribed instead and the transaction commits.	
T5 Sequences	An appointment insert is rolled back but the identity sequence still advances.	Sequences are deliberately non-transactional, so ids have gaps and must never be used as counts

psql - waynes_clinic - T1 COMMIT and T2 ROLLBACK

```
T1  COMMIT - register a walk-in patient AND book her first visit as one unit
================================================================
BEGIN
 txid
------
 3900

INSERT 0 1
Inside the transaction both rows exist:
    mrn     |    patient     | appointment_id |  status   |       slot
------------+----------------+----------------+-----------+------------------
 WCB0099001 | Renee Calloway |         591714 | Scheduled | Tue 03 Nov 09:20

COMMIT
After COMMIT, a brand-new connection (\c) can see them - the change is durable and visible:
You are now connected to database "waynes_clinic" as user "postgres".
SET
SET
    mrn     | created_by_txid | appointment_id |  status
------------+-----------------+----------------+-----------
 WCB0099001 |            3900 |         591714 | Scheduled

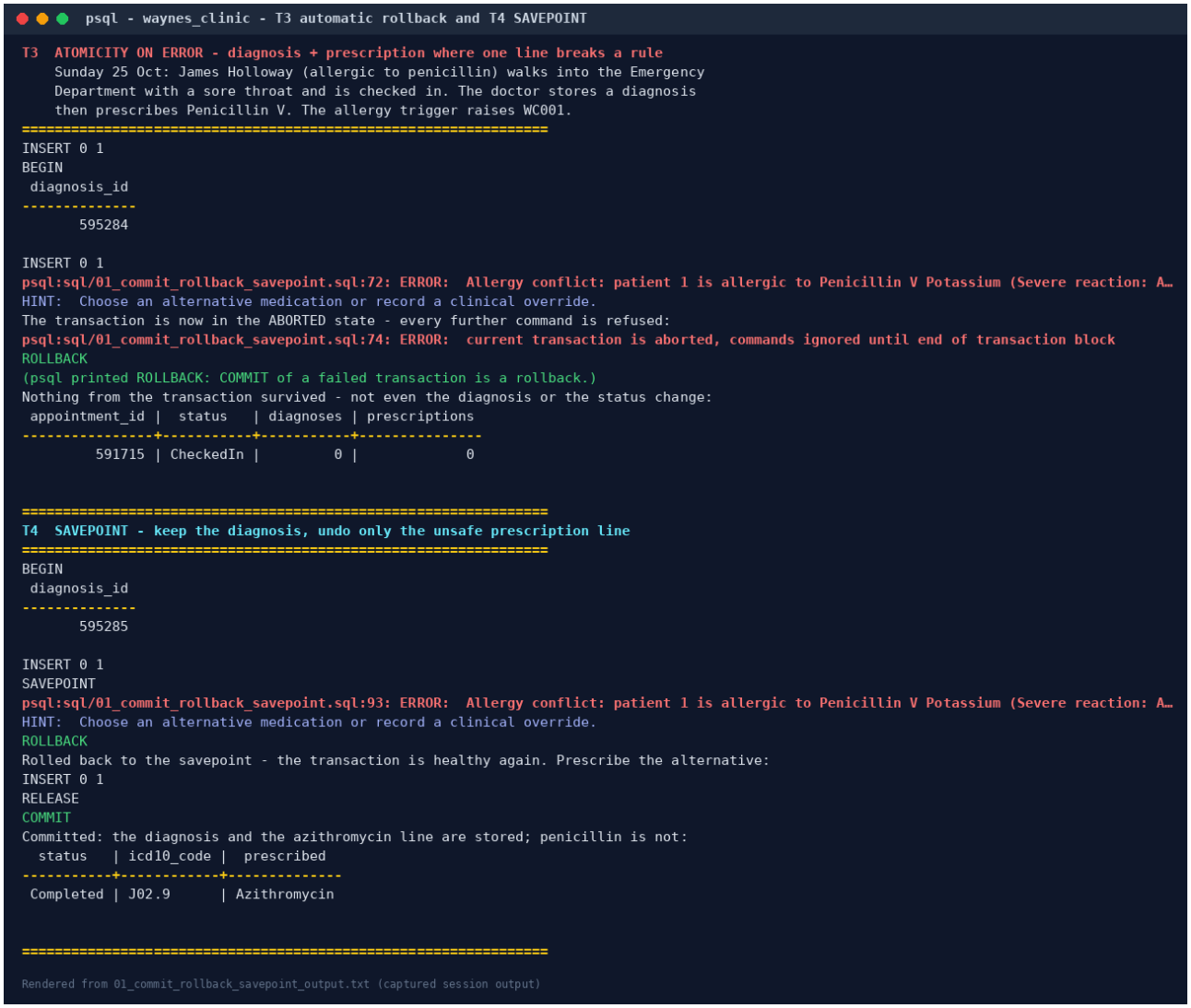

================================================================
T2  EXPLICIT ROLLBACK - a pharmacist keys the wrong quantity and cancels
================================================================
   generic_name   | stock_before
------------------+--------------
 Insulin Glargine |        41892

BEGIN
UPDATE 1
INSERT 0 1
Uncommitted state seen by this session:
 stock_inside_txn | movements_inside_txn
------------------+----------------------
            41792 |                    1

ROLLBACK
After ROLLBACK both the stock change and the audit row are gone:
 stock_after_rollback | movements_after_rollback
----------------------+--------------------------
                41892 |                        0


================================================================

Rendered from 01_commit_rollback_savepoint_output.txt (captured session output)
```



3. Isolation levels and read anomalies

The SQL standard defines four isolation levels by the anomalies each permits. PostgreSQL implements three distinct behaviours, and these demos test each claim.

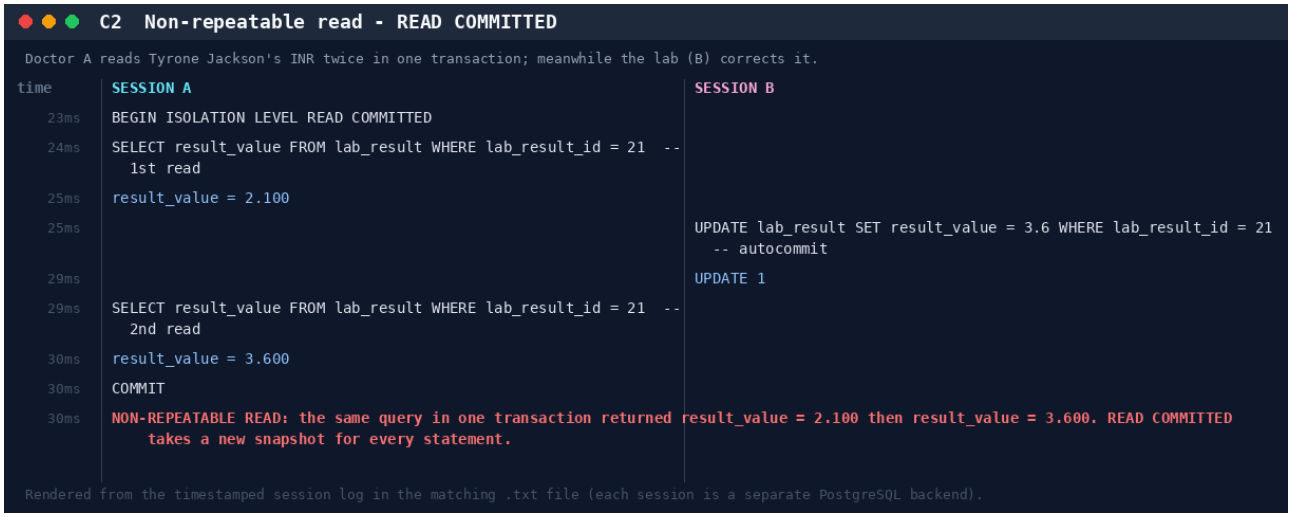
C1 — Dirty read (READ UNCOMMITTED)

The lab scientist (A) changes a potassium result to 6.9 but does not commit. The doctor (B), at `READ UNCOMMITTED`, reads the result and sees **3.3**, the committed value. PostgreSQL treats READ UNCOMMITTED as READ COMMITTED, so **dirty reads are impossible at every level**.

C2 — Non-repeatable read

Doctor A reads Tyrone Jackson's INR twice in one transaction while the lab (B) corrects it from 2.1 to 3.6 and commits.

Isolation	A's 1st read	A's 2nd read	Anomaly
READ COMMITTED	2.1	**3.6**	**Yes**: a new snapshot is taken for every statement
REPEATABLE READ	2.1	2.1	No: one snapshot for the whole transaction



C3 — Phantom read

A manager counts Dr Raman's bookings for a day twice while a receptionist books a new slot.

Isolation	1st count	2nd count	Anomaly
READ COMMITTED	n	**n + 1**	**Yes**: a new row appeared
REPEATABLE READ	n	n	No: PostgreSQL's repeatable read is snapshot isolation, which also stops phantoms (stricter than the standard requires)

4. Concurrency problems and control mechanisms

C4 — Lost update: two pharmacists dispense the same drug

Insulin stock is 10. Pharmacist A dispenses 3 and B dispenses 4, so the correct final stock is **3**.

Variant	Mechanism	Final stock	Evidence
C4a read value → compute in app → write back (READ COMMITTED)	none	**6 — LOST UPDATE**	log
C4b `SELECT … FOR UPDATE`	pessimistic row lock: B waited **410 ms** until A committed, then read 7	3	log
C4c `UPDATE … SET stock = stock - n`	atomic update: B queued on A's row lock, then re-evaluated against the new version	3	log
C4d same code at REPEATABLE READ	first-updater-wins: B got `40001 could not serialize access due to concurrent update`, retried and succeeded	3	log
C4e compare-and-set `UPDATE … WHERE stock_on_hand = <value read>`	optimistic: B's update matched 0 rows, so it re-read and retried; no locks held while the user thinks	3	log

C4a Lost update - read-modify-write at READ COMMITTED

Two pharmacists dispense insulin at the same time: A takes 3 pens, B takes 4. Stock starts at 10.

time	SESSION A	SESSION B
24ms	BEGIN	
24ms		BEGIN
25ms	SELECT stock_on_hand FROM medication WHERE generic_name = 'Insulin Glargine'	
25ms	stock_on_hand = 10	
26ms		SELECT stock_on_hand FROM medication WHERE generic_name = 'Insulin Glargine'
26ms		stock_on_hand = 10
26ms	UPDATE medication SET stock_on_hand = 7 -- 10 - 3, computed by the application	
27ms	UPDATE 1	
27ms	COMMIT	
28ms		UPDATE medication SET stock_on_hand = 6 -- 10 - 4, based on its stale read
29ms		UPDATE 1
29ms		COMMIT
30ms	LOST UPDATE: final stock is 6, but 7 pens were dispensed from 10 so it must be 3. A's update was silently overwritten.	

Rendered from the timestamped session log in the matching .txt file (each session is a separate PostgreSQL backend).

C4b Fix 1 - pessimistic locking with SELECT ... FOR UPDATE

Same race, but each pharmacist locks the medication row when reading it.

time	SESSION A	SESSION B
34ms	BEGIN	
35ms		BEGIN
35ms	SELECT stock_on_hand FROM medication WHERE ... FOR UPDATE	
36ms	stock_on_hand = 10	
36ms		SELECT stock_on_hand FROM medication WHERE ... FOR UPDATE
437ms		... BLOCKED - waiting for a lock held by another session
444ms	pg_stat_activity + pg_locks: A: pid 8501, idle in transaction, wait=Client/ClientRead, blocked_by=- | SELECT stock_on_hand FROM medication WHERE medication_id = $ B: pid 8502, active, wait=Lock/transactionid, blocked_by=A | SELECT stock_on_hand FROM medication WHERE medication_id = $ lock WAITING: B ShareLock on transactionid 3923 lock GRANTED: A ExclusiveLock on transactionid 3923	
444ms	UPDATE medication SET stock_on_hand = 7 -- 10 - 3	
445ms	UPDATE 1	
445ms	COMMIT	
447ms		stock_on_hand = 7 (unblocked after 410 ms)
447ms		UPDATE medication SET stock_on_hand = 3 -- 7 - 4
448ms		UPDATE 1
448ms		COMMIT
449ms	Final stock 3. B was made to wait until A committed, then read the new value (7).	

Rendered from the timestamped session log in the matching .txt file (each session is a separate PostgreSQL backend).

The monitor session in C4b reads `pg_stat_activity` and `pg_locks` while B is blocked. It shows B `waiting on Lock/transactionid`, `blocked_by = A`: the lock is visible inside the server, not just inferred from timing.

Which fix should the clinic use? The atomic `UPDATE` (C4c) is simplest when the new value can be computed in SQL. `dispense_prescription()` uses `SELECT … FOR UPDATE` (C4b) because it must check stock *before* deciding to dispense. Optimistic control (C4e) suits web screens where a user may hold a form open for minutes.

C5 — Write skew: the on-call rule

Rule: at least one doctor must stay on call. Dr Fairbanks (A) and Dr Whitfield (B) each check that 2 doctors are on call, and each then signs *themselves* off. The two transactions update **different rows**, so there is no write-write conflict for row locks to catch.

Isolation	Outcome
REPEATABLE READ	Both commit. **0 doctors on call: the invariant is broken** (write skew is allowed by snapshot isolation)
SERIALIZABLE	B fails with `40001 could not serialize access due to read/write dependencies among`

Isolation	Outcome
	`transactions`. On retry B sees only 1 doctor on call and is refused. **1 doctor remains on call.**

C5 Write skew - the on-call rule - REPEATABLE READ

Rule: at least one doctor must stay on call. Dr Fairbanks (A) and Dr Whitfield (B) each check that 2 are on call, then each signs off.

time	SESSION A	SESSION B
26ms	BEGIN ISOLATION LEVEL REPEATABLE READ	
26ms		BEGIN ISOLATION LEVEL REPEATABLE READ
26ms	SELECT count(*) FROM on_call_roster WHERE on_call -- rule check	
28ms	on_call_now = 2	
28ms		SELECT count(*) FROM on_call_roster WHERE on_call -- rule check
29ms		on_call_now = 2
29ms	UPDATE on_call_roster SET on_call = false WHERE doctor_id = 9	
30ms	UPDATE 1	
30ms		UPDATE on_call_roster SET on_call = false WHERE doctor_id = 4
30ms		UPDATE 1
30ms	COMMIT	
31ms		COMMIT
33ms	WRITE SKEW: both checks passed against the same snapshot, each updated a DIFFERENT row, so no write-write conflict was seen. Nobody is on call - the invariant is broken.	

Rendered from the timestamped session log in the matching .txt file (each session is a separate PostgreSQL backend).

C5 Write skew - the on-call rule - SERIALIZABLE

Rule: at least one doctor must stay on call. Dr Fairbanks (A) and Dr Whitfield (B) each check that 2 are on call, then each signs off.

time	SESSION A	SESSION B
24ms	BEGIN ISOLATION LEVEL SERIALIZABLE	
24ms		BEGIN ISOLATION LEVEL SERIALIZABLE
25ms	SELECT count(*) FROM on_call_roster WHERE on_call -- rule check	
26ms	on_call_now = 2	
26ms		SELECT count(*) FROM on_call_roster WHERE on_call -- rule check
27ms		on_call_now = 2
27ms	UPDATE on_call_roster SET on_call = false WHERE doctor_id = 9	
28ms	UPDATE 1	
28ms		UPDATE on_call_roster SET on_call = false WHERE doctor_id = 4
28ms		UPDATE 1
28ms	COMMIT	
29ms		COMMIT
30ms		ERROR 40001: could not serialize access due to read/write dependencies among transactions
30ms		ROLLBACK
30ms	B's application retries after SQLSTATE 40001:	
30ms		BEGIN ISOLATION LEVEL SERIALIZABLE
30ms		SELECT count(*) FROM on_call_roster WHERE on_call -- re-check
31ms		on_call_now = 1
31ms	Only 1 doctor on call -> the application refuses B's request and ends the transaction.	
31ms		ROLLBACK
31ms	1 doctor still on call. Serializable Snapshot Isolation tracked the read/write dependencies and aborted B (40001); the retry saw the truth.	

Rendered from the timestamped session log in the matching .txt file (each session is a separate PostgreSQL backend).

PostgreSQL's SERIALIZABLE level uses **Serializable Snapshot Isolation (SSI)**. It records which rows each transaction read (predicate locks) and aborts one transaction when a dangerous cycle of read/write dependencies appears. The application must be ready to retry on `40001`. That is the price of a guarantee no `CHECK` constraint can express.

C6 — Double booking: application check vs database constraint

Variant	Outcome
C6a "Is the slot free? Then insert" on a table with no constraint	Both receptionists see 0 clashes and both insert: **2 patients in one slot**
C6b `book_appointment()` on the real table (EXCLUDE constraint)	B **blocks** (it cannot know if the slot is taken until A finishes). A commits, and B immediately gets `23P01 Slot taken`
C6c Same, but A rolls back	B's booking succeeds once A's tentative row disappears

Variant	Outcome

C6a Double booking - 'check, then insert' in application code

Two receptionists book Dr Okafor's Wed 04 Nov 09:00 slot at the same moment, using a table with no exclusion constraint.

time	SESSION A	SESSION B
22ms	BEGIN	
22ms		BEGIN
22ms	SELECT count(*) FROM appointment_unsafe WHERE doctor_id = 1 AND <overlaps slot>	
23ms	clashes = 0	
23ms		SELECT count(*) FROM appointment_unsafe WHERE doctor_id = 1 AND <overlaps slot>
24ms		clashes = 0
24ms	Both see 0 clashes, so both applications decide the slot is free.	
24ms	INSERT INTO appointment_unsafe ... patient 5	
26ms	INSERT 0 1	
26ms		INSERT INTO appointment_unsafe ... patient 7
28ms		INSERT 0 1
28ms	COMMIT	
29ms		COMMIT
30ms	DOUBLE BOOKING: 2 patients now hold the same slot. Neither transaction could see the other's uncommitted insert, and nothing in the database forbade the result.	

Rendered from the timestamped session log in the matching .txt file (each session is a separate PostgreSQL backend).

C6b Double booking prevented by the EXCLUDE constraint

Same race on the real appointment table via book_appointment(). Slot: Dr Okafor Wed 04 Nov 09:00.

time	SESSION A	SESSION B
34ms	BEGIN	
34ms		BEGIN
34ms	SELECT book_appointment(patient 5, doctor 1, 'Wed 04 Nov 09:00')	
39ms	appointment_id = 591719	
39ms		SELECT book_appointment(patient 7, doctor 1, 'Wed 04 Nov 09:00')
439ms		... BLOCKED - waiting for a lock held by another session
444ms	pg_stat_activity + pg_locks: A: pid 8519, idle in transaction, wait=Client/ClientRead, blocked_by=- | SELECT book_appointment(5, 1, $1, 'Receptionist A') AS appoi B: pid 8520, active, wait=Lock/transactionid, blocked_by=A | SELECT book_appointment(7, 1, $1, 'Receptionist B') AS appoi lock WAITING: B ShareLock on transactionid 3945 lock GRANTED: A ExclusiveLock on transactionid 3945 lock GRANTED: B ExclusiveLock on transactionid 3947 lock GRANTED: B ExclusiveLock on transactionid 3948	
444ms	B cannot know yet whether the slot is taken: it depends on whether A commits. It waits on A's transaction id.	
444ms	COMMIT	
446ms		ERROR 23P01: Slot taken: doctor 1 already has an appointment at 2026-11-04 09:00:00 (after 407 ms)
446ms		ROLLBACK
447ms	1 booking for the slot. The constraint is enforced by the GiST index itself, so the check and the insert are one atomic step - there is no race window.	

Rendered from the timestamped session log in the matching .txt file (each session is a separate PostgreSQL backend).

This is the strongest argument for declaring rules in the schema (Portfolio 1). The exclusion constraint makes "check" and "insert" one atomic step inside the index, so there is no race window. Waiting instead of failing fast (C6c) is also correct: the conflicting row might never commit.

C7 — Deadlock: detection and prevention

A moves stock insulin → albuterol while B moves albuterol → insulin. Each locks its first row, then waits for the other's: a cycle.

- **C7a.** After `deadlock_timeout` (1 s), PostgreSQL's deadlock detector found the cycle and aborted A with `40P01 deadlock detected`; B completed. The server log records both processes and the lock each was waiting for (included in the evidence).
- **C7b. Prevention.** Both transfers first lock *both* rows with `ORDER BY medication_id FOR UPDATE`. A consistent global lock order turns the cycle into a queue: B waited about 400 ms and both committed. `dispense_prescription()` applies the same rule.

C7a Deadlock - two transfers lock the same rows in opposite order

A moves stock insulin -> albuterol; B moves albuterol -> insulin. deadlock_timeout = 1s.

time	SESSION A	SESSION B
30ms	BEGIN	
30ms		BEGIN
31ms	UPDATE medication ... WHERE generic_name = 'Insulin Glargine' -- locks row 1	
32ms	UPDATE 1	
32ms		UPDATE medication ... WHERE generic_name = 'Albuterol' -- locks row 2
33ms		UPDATE 1
33ms	UPDATE medication ... WHERE generic_name = 'Albuterol' -- needs row 2	
434ms	... BLOCKED - waiting for a lock held by another session	
440ms	pg_stat_activity + pg_locks: A: pid 8526, active, wait=Lock/transactionid, blocked_by=B | UPDATE medication SET stock_on_hand = stock_on_hand + 5 WHER B: pid 8527, idle in transaction, wait=Client/ClientRead, blocked_by=- | UPDATE medication SET stock_on_hand = stock_on_hand - 5 WHER lock WAITING: A ShareLock on transactionid 3958 lock GRANTED: A ExclusiveLock on transactionid 3957 lock GRANTED: B ExclusiveLock on transactionid 3958	
440ms		UPDATE medication ... WHERE generic_name = 'Insulin Glargine' -- needs row 1: cycle!
841ms		... BLOCKED - waiting for a lock held by another session
1034ms	ERROR 40P01: deadlock detected (after 1,001 ms)	
1035ms		UPDATE 1 (unblocked after 594 ms)
1035ms	ROLLBACK	
1035ms		COMMIT
1037ms	PostgreSQL server log: [8526] postgres@waynes_clinic ERROR: deadlock detected [8526] postgres@waynes_clinic DETAIL: Process 8526 waits for ShareLock on transaction 3958; blocked by process 8527. Process 8527 waits for ShareLock on transaction 3957; blocked by process 8526. Process 8526: UPDATE medication SET stock_on_hand = stock_on_hand + 5 WHERE medication_id = $1 Process 8527: UPDATE medication SET stock_on_hand = stock_on_hand + 5 WHERE medication_id = $1	
1037ms	PostgreSQL's deadlock detector found the wait-for cycle after deadlock_timeout and aborted session A (40P01) so that B could finish. A's work is lost and must be retried.	

Rendered from the timestamped session log in the matching .txt file (each session is a separate PostgreSQL backend).

5. MVCC and durability

C8 — Row versions. A opens a REPEATABLE READ snapshot and reads insulin stock 10 at physical location `ctid (0,72)`, created by transaction 3961. B updates the stock to 99 **without waiting** and creates a new version at `(0,73)`. The old version's `xmax` is now B's transaction id, yet A, inside its snapshot, still reads 10 from `(0,72)`, and a fresh session C sees 99 at `(0,73)`. Two versions of one row coexist until `VACUUM` removes the dead one.

C8 MVCC - readers never block writers, writers never block readers

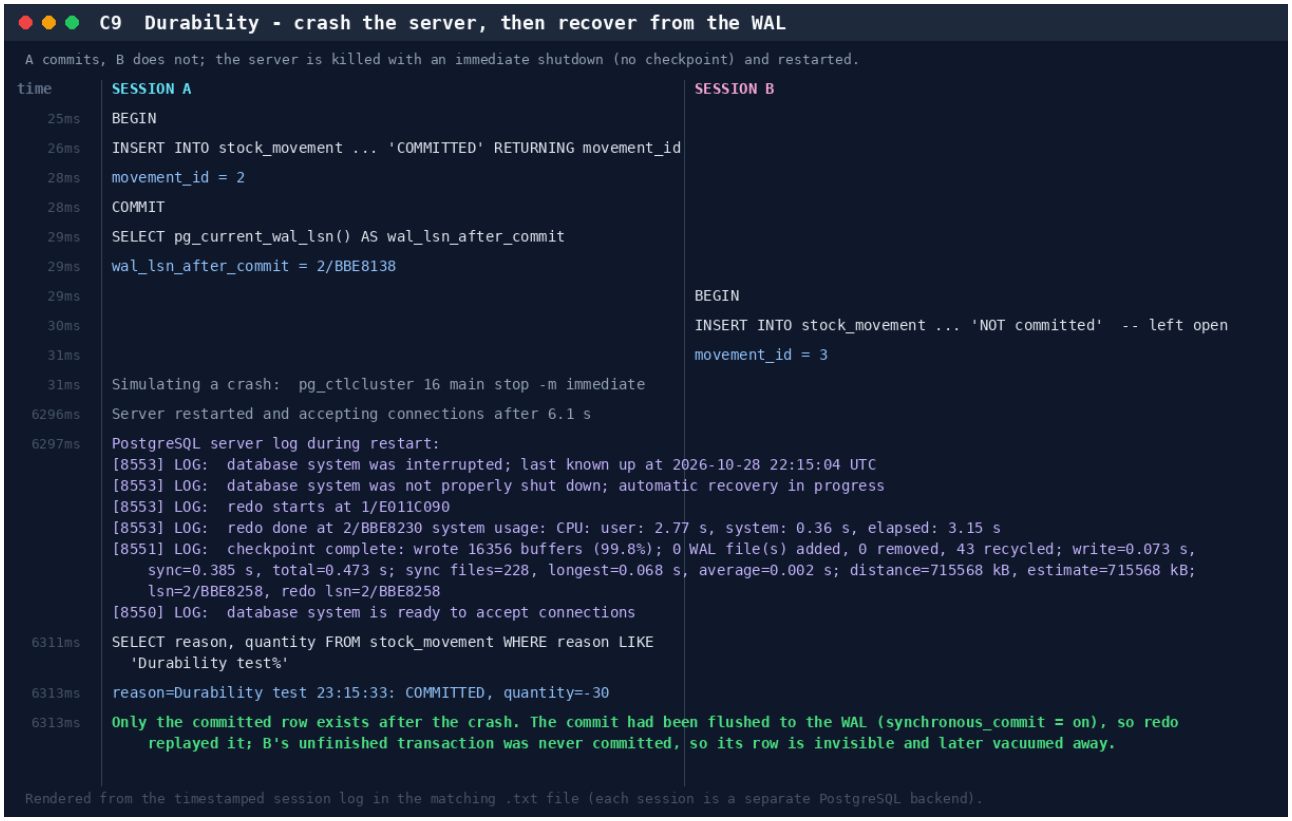
A holds a REPEATABLE READ snapshot open; B updates and commits; A still sees the old row version.

time	SESSION A	SESSION B
42ms	BEGIN ISOLATION LEVEL REPEATABLE READ	
43ms	SELECT ctid, xmin, xmax, stock_on_hand FROM medication WHERE ...	
44ms	ctid=(0,72), xmin=3961, xmax=0, stock_on_hand=10	
44ms		UPDATE medication SET stock_on_hand = 99 WHERE ... RETURNING ctid, xmin -- no waiting
48ms		ctid=(0,73), xmin=3962
48ms	SELECT ctid, xmin, xmax, stock_on_hand ... -- same snapshot	
48ms	ctid=(0,72), xmin=3961, xmax=3962, stock_on_hand=10	
48ms	Session C (a new snapshot) sees the version B created:	
48ms		[C] SELECT ctid, xmin, xmax, stock_on_hand ... -- new snapshot
49ms		[C] ctid=(0,73), xmin=3962, xmax=0, stock_on_hand=99
49ms	COMMIT	
50ms	Two physical versions of the row existed at once (different ctid). The old version's xmax was set to B's transaction id, but A's snapshot predates B, so A kept reading it. VACUUM later removes it.	

Rendered from the timestamped session log in the matching .txt file (each session is a separate PostgreSQL backend).

C9 — Durability: a real crash test. Session A commits a stock movement; session B inserts another row but never commits. The server is then killed with `pg_ctl stop -m immediate` (no checkpoint, equivalent to a power cut) and restarted. The server log shows *"database system was not properly shut down; automatic recovery in progress … redo starts … redo done"*. Afterwards **only the committed row exists**. `COMMIT` returned only after the

commit record was flushed to the write-ahead log (`synchronous_commit = on`), so WAL replay restored it. B's transaction never committed, so its row is invisible.



6. Results matrix

What was **observed** on PostgreSQL 16 (`evidence/concurrency_summary.md`):

Anomaly	READ COMMITTED (default)	REPEATABLE READ	SERIALIZABLE
Dirty read	prevented (C1)	prevented	prevented
Non-repeatable read	**occurs** (C2)	prevented (C2)	prevented
Phantom read	**occurs** (C3)	prevented (C3)	prevented
Lost update (read-modify-write)	**occurs** (C4a)	detected → `40001` (C4d)	detected → `40001`
Write skew	**occurs**	**occurs** (C5)	detected → `40001` (C5)

Recommendations for Wayne's Clinic.

1. Keep READ COMMITTED as the default for its throughput, and make every multi-row read-modify-write either an atomic `UPDATE` or a `SELECT … FOR UPDATE`.
2. Use SERIALIZABLE, with automatic retry on `40001`, for the few transactions that protect a cross-row invariant: the on-call roster, bed allocation and controlled-drug registers.
3. Enforce booking rules with constraints, not application checks.
4. Lock rows in a fixed order to avoid deadlocks, and treat `40P01` as retryable.
5. Keep `synchronous_commit = on` for clinical data.

7. Reproduce

```
./run_portfolio3.sh                # needs Portfolio 1 loaded; Portfolio 2 data optional
SKIP_CRASH_TEST=1 ./run_portfolio3.sh   # skip the server restart on shared machines
```

`scripts/session_lab.py` is a small harness: each `Session` is its own connection, `.start()` sends a statement expected to block in a background thread, and `Monitor.snapshot()` reads `pg_stat_activity`/`pg_locks` while it waits. The timelines are drawn from the same timestamped events as the `.txt` logs.